

Appendix B. Evaluate Your Microservice

To help the reader evaluate the production-readiness of their microservice(s) and microservice ecosystem, Chapters [3](#)–[7](#) conclude with a short list of questions associated with the production-readiness standard discussed. The questions are organized by topic, and correspond to the sections within each chapter. All of the questions from each chapter have been collected here for easy reference.

Stability and Reliability

The Development Cycle

- Does the microservice have a central repository where all code is stored?
- Do developers work in a development environment that accurately reflects the state of production (e.g., that accurately reflects the real world)?
- Are there appropriate lint, unit, integration, and end-to-end tests in place for the microservice?
- Are there code review procedures and policies in place?
- Is the test, packaging, build, and release process automated?

The Deployment Pipeline

- Does the microservice ecosystem have a standardized deployment pipeline?
- Is there a staging phase in the deployment pipeline that is either full or partial staging?
- What access does the staging environment have to production services?
- Is there a canary phase in the deployment pipeline?
- Do deployments run in the canary phase for a period of time that is long enough to catch any failures?

- Does the canary phase accurately host a random sample of production traffic?
- Are the microservice's ports the same for canary and production?
- Are deployments to production done all at the same time, or incrementally rolled out?
- Is there a procedure in place for skipping the staging and canary phases in case of an emergency?

Dependencies

- What are this microservice's dependencies?
- What are its clients?
- How does this microservice mitigate dependency failures?
- Are there backups, alternatives, fallbacks, or defensive caching for each dependency?

Routing and Discovery

- Are health checks to the microservice reliable?
- Do health checks accurately reflect the health of the microservice?
- Are health checks run on a separate channel within the communication layer?
- Are there circuit breakers in place to prevent unhealthy microservices from making requests?
- Are there circuit breakers in place to prevent production traffic from being sent to unhealthy hosts and microservices?

Deprecation and Decommissioning

- Are there procedures in place for decommissioning a microservice?
- Are there procedures in place for deprecating a microservice's API endpoints?

Scalability and Performance

Knowing the Growth Scale

- What is this microservice's qualitative growth scale?

- What is this microservice's quantitative growth scale?

Efficient Use of Resources

- Is the microservice running on dedicated or shared hardware?
- Are any resource abstraction and allocation technologies being used?

Resource Awareness

- What are the microservice's resource requirements (CPU, RAM, etc.)?
- How much traffic can one instance of the microservice handle?
- How much CPU does one instance of the microservice require?
- How much memory does one instance of the microservice require?
- Are there any other resource requirements that are specific to this microservice?
- What are the resource bottlenecks of this microservice?
- Does this microservice need to be scaled vertically, horizontally, or both?

Capacity Planning

- Is capacity planning performed on a scheduled basis?
- What is the lead time for new hardware?
- How often are hardware requests made?
- Are any microservices given priority when hardware requests are made?
- Is capacity planning automated or is it manual?

Dependency Scaling

- What are this microservice's dependencies?
- Are the dependencies scalable and performant?
- Will the dependencies scale with this microservice's expected growth?
- Are dependency owners prepared for this microservice's expected growth?

Traffic Management

- Are the microservice's traffic patterns well understood?
- Are changes to the service scheduled around traffic patterns?
- Are drastic changes in traffic patterns (especially bursts of traffic) handled carefully and appropriately?
- Can traffic be automatically routed to other datacenters in case of failure?

Task Handling and Processing

- Is the microservice written in a programming language that will allow the service to be scalable and performant?
- Are there any scalability or performance limitations in the way the microservice handles requests?
- Are there any scalability or performance limitations in the way the microservice processes tasks?
- Do developers on the microservice team understand how their service processes tasks, how efficiently it processes those tasks, and how the service will perform as the number of tasks and requests increases?

Scalable Data Storage

- Does this microservice handle data in a scalable and performant way?
- What type of data does this microservice need to store?
- What is the schema needed for its data?
- How many transactions are needed and/or made per second?
- Does this microservice need higher read or write performance?
- Is it read-heavy, write-heavy, or both?
- Is this service's database scaled horizontally or vertically? Is it replicated or partitioned?
- Is this microservice using a dedicated or shared database?
- How does the service handle and/or store test data?

Fault Tolerance and Catastrophe-Preparedness

Avoiding Single Points of Failure

- Does the microservice have a single point of failure?
- Does it have more than one point of failure?
- Can any points of failure be architected away, or do they need to be mitigated?

Catastrophes and Failure Scenarios

- Have all of the microservice's failure scenarios and possible catastrophes been identified?
- What are common failures across the microservice ecosystem?
- What are the hardware-layer failure scenarios that can affect this microservice?
- What communication-layer and application-layer failures can affect this microservice?
- What sorts of dependency failures can affect this microservice?
- What are the internal failures that could bring down this microservice?

Resiliency Testing

- Does this microservice have appropriate lint, unit, integration, and end-to-end tests?
- Does this microservice undergo regular, scheduled load testing?
- Are all possible failure scenarios implemented and tested using chaos testing?

Failure Detection and Remediation

- Are there standardized processes across the engineering organization(s) for handling incidents and outages?
- How do failures and outages of this microservice impact the business?
- Are there clearly defined levels of failure?
- Are there clearly defined mitigation strategies?
- Does the team follow the five stages of incident response when incidents and outages occur?

Monitoring

Key Metrics

- What are this microservice's key metrics?
- What are the host and infrastructure metrics?
- What are the microservice-level metrics?
- Are all the microservice's key metrics monitored?

Logging

- What information does this microservice need to log?
- Does this microservice log all important requests?
- Does the logging accurately reflect the state of the microservice at any given time?
- Is this logging solution cost-effective and scalable?

Dashboards

- Does this microservice have a dashboard?
- Is the dashboard easy to interpret? Are all key metrics displayed on the dashboard?
- Can I determine whether or not this microservice is working correctly by looking at the dashboard?

Alerting

- Is there an alert for every key metric?
- Are all alerts defined by good, signal-providing thresholds?
- Are alert thresholds set appropriately so that alerts will fire before an outage occurs?
- Are all alerts actionable?
- Are there step-by-step triage, mitigation, and resolution instructions for each alert in the on-call runbook?

On-Call Rotations

- Is there a dedicated on-call rotation responsible for monitoring this microservice?
- Is there a minimum of two developers on each on-call shift?

- Are there standardized on-call procedures across the engineering organization?

Documentation and Understanding

Microservice Documentation

- Is the documentation for all microservices stored in a centralized, shared, and easily accessible place?
- Is the documentation easily searchable?
- Are significant changes to the microservice accompanied by updates to the microservice's documentation?
- Does the microservice's documentation contain a description of the microservice?
- Does the microservice's documentation contain an architecture diagram?
- Does the microservice's documentation contain contact and on-call information?
- Does the microservice's documentation contain links to important information?
- Does the microservice's documentation contain an onboarding and development guide?
- Does the microservice's documentation contain information about the microservice's request flow, endpoints, and dependencies?
- Does the microservice's documentation contain an on-call runbook?
- Does the microservice's documentation contain an FAQ section?

Microservice Understanding

- Can every developer on the team answer questions about the production-readiness of the microservice?
- Is there a set of principles and standards that all microservices are held to?
- Is there an RFC process in place for every new microservice?
- Are existing microservices reviewed and audited frequently?
- Are architecture reviews held for every microservice team?
- Is there a production-readiness audit process in place?

- Are production-readiness roadmaps used to bring the microservice to a production-ready state?
- Do the production-readiness standards drive the organization's OKRs?
- Is the production-readiness process automated?

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)